

Sorokin School



Task-Management System [часть 5]

🔥 Задание к модулю №5 по интенсиву Spring Boot

Сегодня вы переходите к завершающему этапу проекта — подготовке к код-ревью. Это важный момент, когда вы приводите код в порядок, улучшаете архитектуру и повышаете читаемость, чтобы ваш проект выглядел профессионально. Такой подход не только улучшает качество кода, но и помогает развивать навыки работы в команде и использования систем контроля версий (GIT).

Описание

В этом задании вам предстоит провести ревизию кода вашего проекта Task Management. Вы должны убрать избыточный и некачественный код, внедрить дополнительное логирование. Реализовать маппер для преобразования между моделью и сущностью, а также добавить API метод для поиска задач с фильтрацией и пагинацией, используя @Query запросы в репозитории. В конце вам предстоит подготовить проект к код-ревью и оформить его на GitHub.

Знания для выполнения (что вам предстоит закрепить)

- Ревизия и рефакторинг кода: проверка наименования классов, методов и переменных, устранение дублирования и улучшение бизнес-логики.
- Внедрение логирования в ключевых местах приложения (например, вход в методы сервиса и обработка исключений).
- Разработка TaskMapper как компонента для маппинга между бизнес-моделью (Task) и сущностью (TaskEntity).
- Реализация поиска с фильтрацией и пагинацией с использованием Spring Data JPA и аннотации @Query.
- Работа с параметрами запроса через @RequestParam для формирования объекта фильтра (TaskSearchFilter).
- Оформление проекта в системе контроля версий Git, создание ветки, объединение коммитов (squash) и подготовка Pull Request для код-ревью.

Задание на разработку

🔥 Что конкретно детально нужно реализовать

Ревизия и чистка кода:

- Пройдитесь по всему проекту (Task Management) и проверьте корректность наименований классов, методов и переменных.
- Устраните всё, что напоминает «под-объект» или «копирасту». Убедитесь, что логика методов оптимизирована и не содержит дублирования.

Внедрение логирования: добавьте логирование в ключевых местах (если не было сделано на прошлых этапах):

- Вход в метод сервиса.
- Обработка исключений (если отсутствует, добавьте в GlobalExceptionHandler).
- Основные участки бизнес-логики, где это ранее было пропущено.

Реализация TaskMapper:

- Создайте класс `TaskMapper` для преобразования между моделью Task и сущностью TaskEntity.
- Аннотируйте его как `@Component` и используйте его в сервисном слое для конвертации данных.

Добавление метода поиска с фильтром и пагинацией:

- В контроллере создайте API метод, принимающий параметры через `@RequestParam`, формирующий объект фильтра (TaskSearchFilter) и передающий его в сервис.
- В репозитории реализуйте метод поиска задач с использованием @Query, который принимает параметры для фильтрации (по `creatorId`, `assignedUserId`, `status`, `priority`).

Доработка метода /start задачи

- Для того, чтобы проверить, что у пользователя менее 5 задач активных - реализуйте запрос через @Query
- Исключите обход всех задач in-memory, это влияет на производительность

Подготовка к код-ревью и сдача проекта:

- Приведите код в порядок, уберите все лишнее и неиспользуемое.
- Соберите все изменения в один коммит (если у вас несколько коммитов, выполните squash).
- Создайте отдельную ветку от main, в которой будут все изменения.
- Создайте репозиторий на GitHub, свяжите его с локальным репозиторием и загрузите изменения.
- Создайте Pull Request и отправьте его на ревью.

Советы по реализации

Немного заметок о том как лучше реализовать задание

Поиск с фильтрацией:

Правильно сформируйте объект `TaskSearchFilter` в контроллере и передавайте его в сервис. Используйте аннотацию `@Query` для оптимизации запросов к базе данных и исключения обхода in-memory коллекций.

Пример фильтра

```
1 public record TaskSearchFilter(  
2     Long creatorId,  
3     Long assignedUserId,  
4     TaskStatus status,  
5     TaskPriority priority,  
6     int pageSize, // Размер страницы для пагинации  
7     int pageNum // Номер страницы для пагинации  
8 ) {}
```